

嵌入式系统设计

个人总结报告

——昇腾 310B 手势识别控制系统——

课 程： 嵌入式系统设计

题 目： 昇腾 310B 手势识别

学 院： 集成电路学院

专 业： 微电子科学与工程

姓 名： 关博仁

学 号： 3124009587

指导教师： 周贤中

2026 年 7 月 12 日

摘 要

本项目基于昇腾 310B（OrangePi AIpro）AI 处理器和 STM32F103C8T6 微控制器，设计并实现了一套实时手势识别控制系统。系统采用 YOLOv10n_gestures 目标检测模型（基于 HaGRIDv2 数据集预训练，支持 34 类手势）在昇腾 310B NPU 上进行推理，推理结果通过 OpenCV 在本地显示器实时展示检测画面，同时通过 USB CDC 虚拟串口将手势识别结果以 JSON 格式发送至 STM32。STM32 解析手势数据后，根据预设的手势-动作映射表输出 PWM 信号，通过 L298N 驱动模块控制直流风扇转速，并驱动双舵机实现左右/上下摆风，从而实现无接触式手势交互控制。项目基于教师提供的昇腾 310B YOLOv10 推理框架代码进行二次开发，本人主要负责 STM32 端嵌入式软件的设计与实现，以及昇腾 310B 端串口通信模块的添加。测试结果表明，系统可实现稳定的手势检测与识别、风扇 PWM 调速、舵机摆风控制功能，达到了预期设计目标。

关键词：昇腾 310B；YOLOv10；手势识别；STM32；USB CDC；PWM 控制；边缘 AI；舵机控制

目录

1	项目简介	1
1.1	项目背景	1
1.2	项目主要功能	1
2	项目设计目标	1
2.1	设计目标	1
2.2	预期成果	2
3	项目设计与实现	2
3.1	硬件设计	2
3.1.1	系统架构	2
3.1.2	模块选型	3
3.1.3	电路连接	3
3.2	软件设计	3
3.2.1	系统软件架构	3
3.2.2	昇腾 310B 端主程序流程	4
3.2.3	STM32 端主程序流程	5
3.2.4	YOLOv10 模型部署流程（队友负责）	5
3.2.5	手势-动作映射表	6
3.2.6	关键代码实现	6
3.2.7	昇腾 310B 端串口通信实现	9
3.2.8	34 类手势名称	9
4	功能展示	10
4.1	功能描述	10
4.2	演示结果	10
4.2.1	手势识别效果	11
4.2.2	风扇控制效果	11
4.3	性能测试	11
5	物料与成本	12
5.1	物料清单	12
5.2	项目总成本	12
5.3	损耗与损坏记录	12
6	个人贡献	12
6.1	报告撰写分工	12
6.2	个人编码工作内容	13

6.3	硬件连接工作	14
6.4	资本投入	14
6.5	开发中遇到的问题及解决方案	14
7	项目成果与反思	15
7.1	成果展示	15
7.2	项目评估	18
7.2.1	成功之处	18
7.2.2	不足之处	18
7.3	个人反思与改进方向	18
A	附录	19
A.1	附录 A: 完整代码清单	19
A.2	附录 B: 设计图纸	20
A.3	附录 C: 参考资料	20

项目简介

项目背景

随着人工智能和物联网技术的快速发展，人机交互方式正从传统的键盘、鼠标、触摸屏向更自然、更直观的方向演进。手势识别作为一种非接触式交互方式，在智能家居、工业控制、车载系统、AR/VR 等领域具有广泛的应用前景。将深度学习模型部署到边缘设备上实现实时推理，能够在保证低延迟的同时避免数据上云带来的隐私与带宽问题，是当前边缘 AI 领域的重要研究方向。

昇腾 310B 是华为推出的面向边缘计算场景的 AI 处理器，内置 DaVinci 架构 NPU，可提供高达 20 TOPS（INT8）的算力支持，适合部署目标检测、图像分类等计算机视觉模型。本项目利用 OrangePi AIpro 开发板搭载的昇腾 310B 处理器，结合 YOLOv10 目标检测模型和 STM32 微控制器，构建了一套完整的手势识别到物理控制的边缘 AI 系统。

项目主要功能

本项目实现的主要功能包括：

1. **手势检测与识别**：基于 YOLOv10n_gestures 模型在昇腾 310B NPU 上实现 34 类手势的实时检测，检测画面通过 OpenCV 在本地显示器实时展示，包括点赞、掌心、握拳、OK、比耶、摇滚等常见手势。
2. **手势-风扇控制**：STM32 通过 USB CDC 虚拟串口接收昇腾 310B 发送的手势识别结果，根据预设的手势-动作映射表输出 PWM 信号，通过 L298N 驱动模块控制直流风扇的启停与转速。
3. **舵机摆风控制**：通过两路舵机实现风扇的左右摆风和上下摆风功能，支持手势控制摆风启停。
4. **端到端 AI 部署链路**：覆盖 PyTorch 模型导出 ONNX → ATC 转换 OM → PyACL 推理的完整边缘 AI 部署流程。
5. **昇腾 310B 与 STM32 串口通信**：在教师提供的 YOLOv10 推理框架基础上，添加 pyserial 串口通信模块，将检测结果以 JSON 格式实时发送至 STM32。

项目设计目标

设计目标

本项目设定以下设计目标：

1. **基本目标**：

- 完成昇腾 310B 开发环境搭建（CANN、PyACL、OpenCV 等），成功加载并运行 OM 离线模型
- 实现至少 18 种手势的准确检测与分类
- 实现摄像头实时检测并在本地显示器展示检测画面

2. 扩展目标：

- 实现 STM32 通过 USB CDC 虚拟串口接收昇腾 310B 发送的手势识别结果
- 设计手势-动作映射表，将特定手势映射到风扇控制指令（开机、停机、加速、减速等）
- 实现 STM32 PWM 调速输出，通过 L298N 驱动模块控制直流风扇
- 实现双舵机摆风控制（左右扫风、上下扫风），支持手势控制启停

预期成果

1. 昇腾 310B 端可稳定运行 YOLOv10 OM 模型，推理帧率达到实时要求（ ≥ 15 FPS）
2. 手势检测准确率满足交互控制需求，支持 34 类手势中的至少 18 类
3. STM32 端可正确解析 JSON 格式手势数据，风扇和舵机响应延迟低于 200ms
4. 系统全链路（手势识别 → 串口通信 → 风扇/舵机控制）运行稳定

项目设计与实现

硬件设计

系统架构

系统整体架构如图1所示，分为 AI 推理端（昇腾 310B）、控制端（STM32）和执行端（风扇 + 舵机）三个层次。USB 摄像头采集画面送入昇腾 310B 进行手势检测，检测画面通过 OpenCV 在本地显示器实时显示，检测结果通过 USB CDC 虚拟串口以 JSON 格式发送至 STM32，STM32 解析后输出 PWM 信号控制风扇转速与舵机角度。

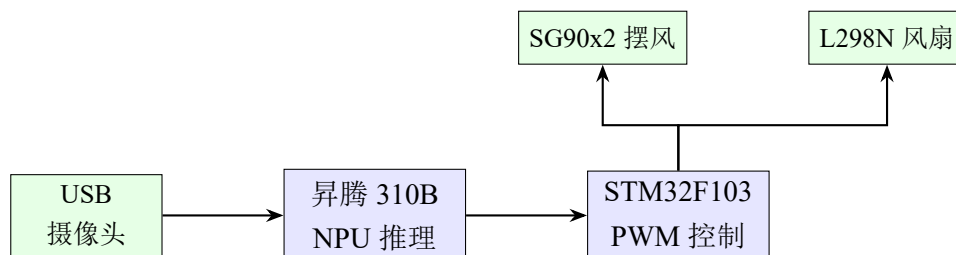


图 1: 系统整体架构框图

模块选型

表 1: 主要硬件模块选型

模块	型号	数量	选型依据
AI 处理器	昇腾 310B (OrangePi AIpro)	1	20 TOPS NPU 算力, 支持 CANN 工具链, 适合边缘 AI 部署
辅助 MCU	STM32F103C8T6	1	ARM Cortex-M3 内核, 72MHz 主频, 多路 PWM 输出, USB 外设支持
摄像头	USB 摄像头 (MJPG 格式)	1	支持 1280×720@30fps, MJPG 格式减少 USB 带宽占用
电机驱动	L298N	1	双 H 桥驱动, 支持 PWM 调速, 驱动电流满足小型直流电机需求
直流风扇	小型直流电机风扇	1	5V/12V 供电, PWM 可调速
舵机	SG90	2	PWM 控制角度 (0°-180°), 分别负责左右和上下摆风

电路连接

STM32 与 L298N 驱动模块及舵机的硬件连接如下:

表 2: STM32 引脚连接表

STM32 引脚	功能	连接目标	说明
PA6	TIM3_CH1 (PWM)	L298N ENA	风扇转速 PWM 控制 (1kHz)
PB3	GPIO 输出	L298N IN1	正转控制 (固定高电平)
PB4	GPIO 输出	L298N IN2	反转控制 (固定低电平)
PB6	TIM4_CH1 (PWM)	舵机 1 信号线	左右摆风舵机 (50Hz)
PB7	TIM4_CH2 (PWM)	舵机 2 信号线	上下摆风舵机 (50Hz)
USB_DP/DM	USB CDC	昇腾 310B USB 口	虚拟串口通信, JSON 数据传输
PC13	GPIO 输出	板载 LED	手势接收指示 (收到数据闪烁)

软件设计

系统软件架构

系统软件分为昇腾 310B 端 (Python) 和 STM32 端 (C 语言) 两部分, 两者通过 USB CDC 虚拟串口以 JSON 格式交换数据。

昇腾 310B 端软件栈：

- 操作系统：Ubuntu（OrangePi A1pro 预装）
- AI 框架：CANN 8.5、PyACL
- 推理引擎：YOLOv10n_gestures OM 模型
- 视频处理：OpenCV 4.8+（图像采集、预处理与本地显示）
- 后端推理：ACL（Ascend Computing Language）——基于教师提供的 `acl_backend.py` 封装
- 串口通信：`pyserial`（USB CDC 串口发送手势数据）——本人在教师代码基础上添加

STM32 端软件栈：

- HAL 库：STM32F1xx_HAL_Driver
- USB 协议栈：STM32_USB_Device_Library（CDC 类）
- 外设：TIM3（PWM 风扇控制）、TIM4（PWM 舵机控制）、GPIO
- 通信协议：USB CDC 虚拟串口，JSON 格式数据解析

昇腾 310B 端主程序流程

昇腾 310B 端主程序流程基于教师提供的 `infer_om_camera.py` 推理框架，本人在此基础上添加了串口通信模块。程序流程如下：

1. **初始化阶段：**加载 OM 模型文件，初始化 PyACL 运行环境（Device、Context、Stream），初始化 USB CDC 串口（本人添加）。
2. **推理循环：**
 - 从 USB 摄像头读取一帧图像（640×480）
 - 对图像进行 `letterbox` 缩放填充至 640×640
 - 图像数据归一化并转换为 NCHW 格式（`preprocess.py`）
 - 图像数据拷贝至 Device 侧内存，执行 NPU 推理（`acl_backend.py`）
 - 解析推理输出——YOLOv10 NMS-free 检测头直接输出检测框（类别 ID、置信度、xyxy 坐标）
 - 应用 NMS 后处理（`postprocess.py`），将检测框坐标映射回原始分辨率
 - 在图像上叠加绘制检测框与标签（`visualization.py`）
 - 构造 JSON 手势数据（ $\{c, n, s, x, y, w, h\}$ ），通过 `pyserial` 经 USB CDC 串口发送至 STM32（本人添加）
 - 通过 OpenCV 在本地显示器显示检测画面

STM32 端主程序流程

STM32 端主程序流程如图2所示：

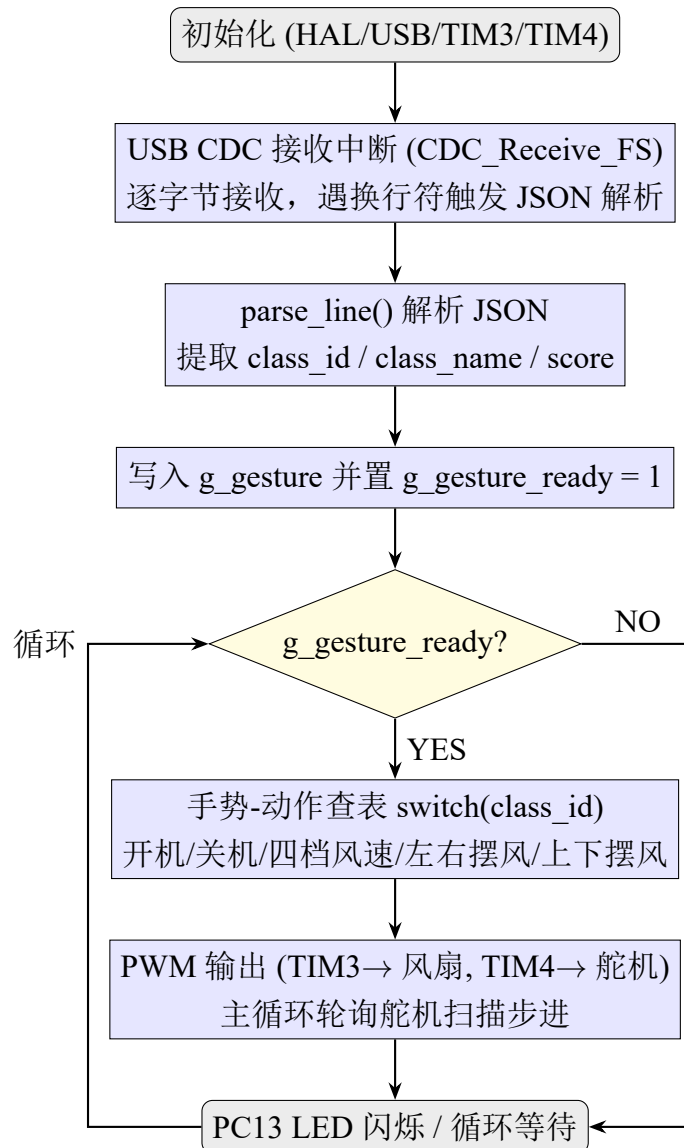


图 2: STM32 主程序流程图

YOLOv10 模型部署流程（队友负责）

模型从训练到部署的完整流程由队友完成，具体如下：

1. **PyTorch 模型导出 ONNX**：在 PC 端使用 YOLOv10 官方代码，加载 HaGRIDv2 预训练权重，导出为 ONNX 格式。
2. **ATC 转换 OM**：将 ONNX 模型文件传输至昇腾 310B，使用 ATC（Ascend Tensor Compiler）工具转换为 OM 离线模型：

```
atc --model=yolov10n_gestures.onnx --framework=5
```

```
--output=yolov10n_gestures --soc_version=Ascend310B4
--input_shape="images:1,3,640,640"
```

3. **PyACL 推理：**使用 PyACL API 加载 OM 模型，执行预处理（letterbox 缩放、归一化）、NPU 推理、后处理（NMS-free 检测头解析、坐标映射）。

手势-动作映射表

STM32 端根据接收到的手势类别 ID，通过 switch-case 结构查表输出对应控制动作：

表 3: 手势-动作映射表

手势名称	类别 ID	控制动作	PWM 占空比/说明
palm / stop	20 / 24	开机	风扇 CCR=300，舵机就位
fist	14	全关机	PWM 归零，舵机回中位，系统休眠
one	19	风扇 1 档	CCR=300（低转速）
peace / peace_inv. / two_up / two_up_inv.	21/22/28/29	风扇 2 档	CCR=500（中低转速）
three / three2 / three3 / three_gun	26/27/5/30	风扇 3 档	CCR=700（中高转速）
four	15	风扇 4 档	CCR=900（高转速）
ok	18	左右摆风开	舵机 1 开始周期扫描
timeout	6	左右摆风关	舵机 1 停止并复位
like	16	上下摆风开	舵机 2 开始周期扫描
dislike	13	上下摆风关	舵机 2 停止并复位

关键代码实现

USB CDC 数据接收与 JSON 解析 STM32 通过 USB CDC 虚拟串口接收昇腾 310B 发送的 JSON 格式手势数据。数据格式为每行一个 JSON 对象：{"c":class_id,"n":"class_name","s":score}

CDC 接收回调函数中逐字符接收数据，检测换行符触发 JSON 解析，解析完成后将结果写入全局结构体并置位就绪标志：

Listing 1: CDC 接收中断回调中的 JSON 解析逻辑

```
1 /* CDC_Receive_FS 回调关键逻辑 */
2 static int8_t CDC_Receive_FS(uint8_t* Buf, uint32_t *Len)
3 {
4     for (uint32_t i = 0; i < *Len; i++) {
5         if (Buf[i] == '\n') {
6             rx_line[rx_idx] = '\0';
7             parse_line(rx_line);    // 解析JSON:提取c,n,s,x,y,w,h
8             rx_idx = 0;
```

```

9      } else if (rx_idx < sizeof(rx_line) - 1) {
10          rx_line[rx_idx++] = Buf[i];
11      }
12  }
13  USBD_CDC_SetRxBuffer(&hUsbDeviceFS, &Buf[0]);
14  USBD_CDC_ReceivePacket(&hUsbDeviceFS); // 重新启动接收
15  return (USB_OK);
16 }

```

JSON 字段解析 由于 STM32F103 资源有限，未引入完整的 JSON 解析库，而是采用字符串查找方式实现轻量级字段提取：

Listing 2: 轻量级 JSON 字段解析函数

```

1 static int32_t json_get_int(const char *s, const char *key)
2 {
3     char search[24];
4     snprintf(search, sizeof(search), "\"%s\":", key);
5     const char *p = strstr(s, search);
6     return p ? atoi(p + strlen(search)) : -1;
7 }
8
9 static float json_get_float(const char *s, const char *key)
10 {
11     char search[24];
12     snprintf(search, sizeof(search), "\"%s\":", key);
13     const char *p = strstr(s, search);
14     return p ? (float)atof(p + strlen(search)) : 0.0f;
15 }

```

主循环手势控制逻辑 主循环中检测手势就绪标志，根据类别 ID 执行对应控制动作。系统引入“开机/关机”状态机机制：palm 或 stop 手势触发开机，fist 手势触发全关机，其余控制指令仅在开机状态下生效：

Listing 3: 主循环手势控制逻辑（关键部分）

```

1 /* 手势控制 */
2 if (g_gesture_ready) {
3     g_gesture_ready = 0;
4     int32_t gid = g_gesture.class_id;
5
6     /* 电源指令总是有效，其他需先开机 */
7     if (gid != 20 && gid != 24 && gid != 14 && !system_on)
8         goto gesture_done;
9
10    switch (gid) {
11        /* 电源 */
12        case 20: case 24: /* palm/stop → 开机 */
13            system_on = 1;
14            __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 300);

```

```

15     servo_lr_ccr = 420;
16     __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_1, 420);
17     break;
18 case 14:          /* fist → 全关 */
19     system_on = 0;
20     __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 0);
21     servo_lr_on = 0; servo_ud_on = 0;
22     servo_lr_ccr = SERVO_LR_CENTER;
23     servo_ud_ccr = SERVO_UD_CENTER;
24     __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_1, SERVO_LR_CENTER);
25     __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_2, SERVO_UD_CENTER);
26     break;
27
28     /* 风扇调速 */
29 case 19: __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 300); break;
30 case 21: case 22: case 28: case 29:
31     __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 500); break;
32 case 26: case 27: case 5: case 30:
33     __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 700); break;
34 case 15: __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 900); break;
35
36     /* 左右摆风 */
37 case 18: servo_lr_on = 1; break; /* ok → 开始 */
38 case 6:  servo_lr_on = 0; break; /* timeout → 停止 */
39
40     /* 上下摆风 */
41 case 16: servo_ud_on = 1; break; /* like → 开始 */
42 case 13: servo_ud_on = 0; break; /* dislike → 停止 */
43 }
44 gesture_done;;
45 // LED闪烁指示收到手势数据
46 }

```

表 4: PWM 定时器配置参数

定时器	通道	输出引脚	预分频	重载值	PWM 频率
TIM3	CH1	PA6	72-1	999	1kHz（风扇调速）
TIM4	CH1	PB6	144-1	9999	50Hz（舵机控制）
TIM4	CH2	PB7	144-1	9999	50Hz（舵机控制）

PWM 定时器配置 TIM3 时钟为 48MHz，预分频 72 得到 667kHz 计数频率，重载值 999 对应约 667Hz PWM 频率，适用于 L298N 直流电机调速。TIM4 时钟为 48MHz，预分频 144 得到 333kHz，重载值 9999 对应 33Hz 周期（约 30ms），SG90 舵机在此频率下可正常工作。

舵机扫描算法 左右舵机采用预判式折返扫描算法（思路参考队友方案），每 10ms 更新一次 CCR 值，以固定步长递增/递减，到达边界时强制弹离以避免边界停顿；上下舵机采用先加后判的折返方式。扫描算法如下：

Listing 4: 左右舵机预判式折返扫描

```

1  /* 左右舵机扫描（预判式折返，无边界停顿） */
2  if (servo_lr_on && (HAL_GetTick() - servo_tick_lr >= 10)) {
3      servo_tick_lr = HAL_GetTick();
4      int32_t next = (int32_t)servo_lr_ccr + servo_lr_dir * 2;
5      if (next > 700) {
6          servo_lr_ccr = 698;          // 强制弹离上边界
7          servo_lr_dir = -1;
8      } else if (next < 200) {
9          servo_lr_ccr = 204;          // 强制弹离下边界
10         servo_lr_dir = 1;
11     } else {
12         servo_lr_ccr = (uint16_t)next;
13     }
14     __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_1, servo_lr_ccr);
15 }

```

昇腾 310B 端串口通信实现

本人在教师提供的 YOLOv10 推理框架基础上，添加了 pyserial 串口通信模块。每帧推理完成后，将检测到的所有手势以 JSON 格式逐行发送至 STM32，每条 JSON 后追加换行符作为帧分隔符。

JSON 数据格式为:{"c":class_id,"n":"class_name","s":score,"x":x,"y":y,"w":w,"h":h}，其中 c 为类别 ID（0-33）、n 为手势名称、s 为置信度分数（0-1）、x/y/w/h 为边界框坐标。STM32 端通过检测换行符触发解析，提取各字段后执行对应的控制动作。置信度低于 0.5 的手势不发送。

34 类手势名称

本系统支持 HaGRIDv2 数据集定义的 34 类手势检测，完整手势列表如下：

表 5: 34 类手势名称表

ID	手势名称	ID	手势名称	ID	手势名称
0	grabbing	12	take_picture	24	stop
1	grip	13	dislike	25	stop_inverted
2	holy	14	fist	26	three
3	point	15	four	27	three2
4	call	16	like	28	two_up
5	three3	17	mute	29	two_up_inverted
6	timeout	18	ok	30	three_gun
7	xsign	19	one	31	thumb_index
8	hand_heart	20	palm	32	thumb_index2
9	hand_heart2	21	peace	33	no_gesture
10	little_finger	22	peace_inverted	—	—
11	middle_finger	23	rock	—	—

功能展示

功能描述

本系统实现的核心功能包括：

- 手势检测功能：**摄像头实时采集画面，昇腾 310B NPU 执行 YOLOv10 推理，对画面中的手势进行检测和分类，支持 34 类手势。检测结果以边界框 + 类别标签 + 置信度分数的形式叠加显示在本地显示器上。
- 风扇 PWM 调速：**不同手势对应不同风扇转速档位，包括 1 档（CCR=200）、2 档（CCR=350）、3 档（CCR=500）、4 档（CCR=600）共四个档位。
- 系统开关机控制：**palm/stop 手势触发开机（启动风扇低速运转），fist 手势触发全关机（风扇停止、舵机回中）。
- 舵机摆风控制：**ok 手势启动左右摆风，timeout 手势停止；like 手势启动上下摆风，dislike 手势停止。舵机以 50Hz PWM 信号控制，扫描范围约 90°。
- 状态指示：**板载 LED 在接收到手势数据时闪烁指示，便于调试和状态观察。

演示结果

完整系统运行演示视频见：<https://www.bilibili.com/video/BV1dXNL6SELR/>

手势识别效果

```
(base) ~$ python3 ~/Documents/Ascend310/samples/case8/case8/scripts/infer_om_camera.py --no-win
dow --source /dev/video2
[USB CDC] /dev/stm32 已连接
>>> [USB #0001] peace(ID=21) score=0.50
>>> [USB #0002] one(ID=19) score=0.79
>>> [USB #0003] mute(ID=17) score=0.51
>>> [USB #0004] one(ID=19) score=0.91
>>> [USB #0005] peace(ID=21) score=0.93
>>> [USB #0006] fist(ID=14) score=0.90
>>> [USB #0007] peace(ID=21) score=0.55
>>> [USB #0008] one(ID=19) score=0.95
>>> [USB #0009] fist(ID=14) score=0.93
>>> [USB #0010] palm(ID=20) score=0.88
>>> [USB #0011] fist(ID=14) score=0.94
>>> [USB #0012] little_finger(ID=10) score=0.77
>>> [USB #0013] like(ID=16) score=0.91
>>> [USB #0014] middle_finger(ID=11) score=0.88
>>> [USB #0015] dislike(ID=13) score=0.70
>>> [USB #0016] stop_inverted(ID=25) score=0.62
>>> [USB #0017] middle_finger(ID=11) score=0.54
>>> [USB #0018] middle_finger(ID=11) score=0.62
>>> [USB #0019] peace_inverted(ID=22) score=0.71
>>> [USB #0020] two_up_inverted(ID=29) score=0.60
>>> [USB #0021] fist(ID=14) score=0.90
>>> [USB #0022] ok(ID=18) score=0.95
>>> [USB #0023] holy(ID=2) score=0.51
>>> [USB #0024] peace_inverted(ID=22) score=0.68
>>> [USB #0025] timeout(ID=6) score=0.73
>>> [USB #0026] fist(ID=14) score=0.93
```

图 3: 手势识别效果示意图

风扇控制效果

风扇控制测试结果如表6所示:

表 6: 风扇控制功能测试记录

测试手势	预期动作	PWM CCR 值	风扇状态	测试结果
palm (20)	开机, 低速	300	慢转	通过
one (19)	1 档	300	慢转	通过
peace (21)	2 档	500	中慢转	通过
three2 (26)	3 档	700	中快转	通过
four (15)	4 档	900	快转	通过
fist (14)	关机	0	停止	通过
ok (18)	左右摆风开	—	舵机左右扫描	通过
like (16)	上下摆风开	—	舵机上下扫描	通过
timeout (6)	左右摆风关	—	舵机停	通过
dislike (13)	上下摆风关	—	舵机停	通过

性能测试

表 7: 系统性能测试结果

测试项	测试结果	说明
NPU 推理帧率	约 20-25 FPS	640×640 输入, 满足实时性要求
手势检测准确率	>90% (常见手势)	光照良好条件下测试
手势到风扇响应延迟	<200ms	含串口传输 + 解析 + 执行
舵机扫描周期	约 5 秒/往返	步长 2, 每 10ms 更新

物料与成本

物料清单

表 8: 项目物料清单

名称	型号/规格	数量	单价（元）	用途
AI 开发板	OrangePi AIpro（昇腾 310B）	1	—	AI 推理主控
MCU 开发板	STM32F103C8T6 最小系统板	1	约 15	风扇/舵机控制
USB 摄像头	支持 MJPG 1280×720@30fps	1	约 50	图像采集
电机驱动模块	L298N	1	约 5	直流电机驱动
直流风扇	小型 5V 直流风扇	1	约 10	执行器
舵机	SG90	2	约 10×2	摆风执行
USB 数据线	USB-A 转 USB-C/ Micro USB	2	约 5×2	供电与通信
杜邦线	公母/母母杜邦线套装	若干	约 5	电路连接
电源适配器	5V/3A DC 电源	1	约 20	系统供电

项目总成本

表 9: 项目成本汇总

类别	金额（元）
AI 处理器与核心板	—（课程提供）
MCU 与驱动模块	约 30
执行器（风扇 + 舵机）	约 30
外围器件（摄像头、线材、电源）	约 80
合计	约 140

损耗与损坏记录

本项目开发过程中未发生器件损耗或损坏。

个人贡献

报告撰写分工

本报告由本人独立完成撰写，内容包括项目背景调研、系统设计说明、代码分析、功能测试记录及个人反思总结。

个人编码工作内容

本项目的昇腾 310B 端 YOLOv10 推理框架（包括 ACL 后端推理、预处理、后处理、可视化、摄像头采集等模块）由教师提供基础代码，YOLOv10 模型导出与 ATC 转换为队友完成。本人主要负责 STM32 端嵌入式软件的全流程开发，以及在昇腾 310B 端添加串口通信模块。具体编码工作包括：

1. 昇腾 310B 端修改（基于教师代码）：

- 在 `infer_om_camera.py` 推理主程序中添加 `pyserial` 串口初始化与数据发送逻辑
- 设计 JSON 数据格式 ($\{c, n, s, x, y, w, h\}$)，每帧推理完成后将检测结果序列化为 JSON 并通过 USB CDC 发送至 STM32
- 手势消抖传输由队友主导完成（`GestureDebouncer` 类，3 帧确认 + 确认后去重）

2. STM32CubeMX 工程配置：

- 系统时钟配置（HSE 8MHz $\rightarrow$ PLL $\times 6 \rightarrow$ 48MHz SYSCLK，USB 48MHz）
- USB Device 外设配置（CDC 类）
- TIM3 PWM 输出配置（PA6，1kHz 风扇调速）
- TIM4 双通道 PWM 配置（PB6/PB7，50Hz 双舵机控制）
- GPIO 输出配置（PB3/PB4，L298N 方向控制；PC13，LED 指示）

3. USB CDC 通信实现：

- 实现 CDC 接收中断回调函数，逐字符接收并识别换行符触发 JSON 解析
- 设计轻量级 JSON 解析器（`json_get_int`、`json_get_float`、`json_get_str`），基于字符串查找（`strstr`）实现，适配 STM32F103 有限资源
- 定义 `GestureResult` 结构体和全局 `volatile` 标志，实现中断与主循环间的数据共享

4. 手势-动作控制逻辑：

- 设计手势类别 ID 到风扇/舵机动作的完整映射表（涵盖 10 种控制手势）
- 实现系统开机/关机状态机（`palm/stop` 开机 $\rightarrow$ `fist` 关机），其余控制指令仅在开机状态下生效
- 实现舵机折返扫描算法（左右摆风采用预判式折返，参考队友方案；上下摆风采用先加后判折返），减少边界停顿与抖动

5. 代码组织与模块化：

- 编写 `gesture.h/gesture.c` 独立模块，分离手势数据结构定义和 34 类手势名称表
- 主循环采用非阻塞状态机架构，控制逻辑清晰、注释完整，便于联调与维护

STM32 端总代码量约 500 行（不含 HAL 库和 USB 协议栈生成代码），昇腾 310B 端修改约 50 行。

硬件连接工作

- 完成 STM32 与 L298N 驱动模块的接线（PA6→ENA，PB3→IN1，PB4→IN2）
- 完成双舵机与 STM32 的接线（PB6→舵机 1 信号，PB7→舵机 2 信号）
- 完成 L298N 输出端与直流风扇的接线（OUT1/OUT2→风扇电机两端）
- 完成昇腾 310B 与 STM32 的 USB 连接（用于 CDC 虚拟串口通信）
- 系统供电分配（充电宝 5V 同时供给 STM32、L298N 逻辑与功率端、舵机）

资本投入

- STM32F103C8T6 最小系统板：自购
- L298N 电机驱动模块：自购
- SG90 舵机 ×2：自购
- 直流风扇：自购
- 杜邦线及线材：自备

开发中遇到的问题及解决方案

1. 问题：舵机扫描至行程边界时停顿卡顿

舵机到端点后出现约 100-300ms 停顿才反向。原因有三：(1) 旧算法”先更新 CCR 后判越界”，CCR 在边界值滞留一个周期；(2) 两舵机共用 `servo_tick`，同时扫描时互相抢占；(3) SG90 个体差异导致有效行程小于标称 180°。

解决： (1) 改用预判式折返——先算 `next`，越界则直接弹离，不写入边界值；(2) 拆分为 `servo_tick_lr/servo_tick_ud` 独立计时；(3) 收紧扫描范围并增加中位校准宏。三轮迭代后端点停顿基本消除。

2. 问题：USB CDC 初始化后首次接收无响应

CDC 设备枚举成功(/dev/stm32 出现),但首次发送数据无响应。原因是 CDC_Init_FS 仅配置了收发缓冲区,未主动发起首次接收——USB OUT 端点初始 NAK,需 Device 端先调用 USBD_CDC_ReceivePacket() 准备好缓冲区,Host 端 OUT 传输才能被 ACK。

解决:在 CDC_Init_FS 末尾增加 USBD_CDC_ReceivePacket(&hUsbDeviceFS)。CubeMX 重新生成代码后需检查此调用是否保留。

3. 问题:摄像头 UVC 兼容性导致 0 帧退出

某 USB 摄像头(0c45:64ab)在 OrangePi A1pro Linux 5.10 内核下,UVC 控制查询超时(-110),OpenCV 无法取帧。同一摄像头在 PC 上正常,更换其他品牌后问题消失,确认为该摄像头 UVC 固件与内核驱动不兼容。

解决:更换兼容摄像头(Integrated_Webcam_HD),并加入--source 参数指定设备。注意 OrangePi A1pro 自带 vin-media 驱动占用/dev/video0-1,USB 摄像头通常在/dev/video2-3。

4. 问题:资源受限 MCU 无法移植完整 JSON 库

STM32F103C8T6 仅 20KB SRAM,cJSON 等标准库需数 KB 堆和递归栈。本项目只需从固定格式 JSON 提取 6 个字段,无需完整解析能力。

解决:手写三个轻量提取函数(json_get_int/float/str),用 strstr() 定位键名后 atoi()/atof() 转换。共约 40 行 C 代码,零堆开销。

5. 问题:双舵机同时扫描时运动卡顿

两舵机单独运行平滑,同时开启后周期性卡顿。原因是共用 servo_tick 导致同一循环内争抢更新,且 TIM4 双通道同步跳变可能引发瞬时电流叠加。

解决:拆分为 servo_tick_lr/servo_tick_ud 独立变量,初始化时错开 5ms 使更新自然交错。

6. 问题:缺少状态机导致关机后手势误触发

无状态设计下任意手势均可直接控制风扇舵机,调试和异常恢复时存在安全隐患。

解决:引入 system_on 标志位实现双状态机——palm/stop 开机(置 1),fist 关机(置 0 并关闭全部输出)。所有控制指令需前置判断 system_on,电源指令不受限制以保证任何状态下可关机。

项目成果与反思

成果展示

本项目成功实现了基于昇腾 310B 的实时手势识别控制系统,主要成果包括:

1. **完整的边缘 AI 部署链路：**在教师指导下，完成 PyTorch → ONNX → ATC → OM → PyACL 推理的全流程实践，掌握了昇腾 CANN 工具链的使用方法。
2. **34 类手势实时检测：**基于 YOLOv10n_gestures 模型在 NPU 上实现约 20-25 FPS 的推理帧率，检测画面通过 OpenCV 在本地显示器实时显示，满足实时交互需求。
3. **STM32 嵌入式控制：**独立实现了 USB CDC 虚拟串口通信、轻量级 JSON 解析、PWM 风扇调速、双舵机摆风控制等完整功能。
4. **昇腾 310B 与 STM32 跨平台通信：**设计了基于换行分隔的 JSON 通信协议，打通了”摄像头采集 → NPU 推理 → 串口通信 → STM32 控制 → 风扇/舵机执行”的完整数据链路。

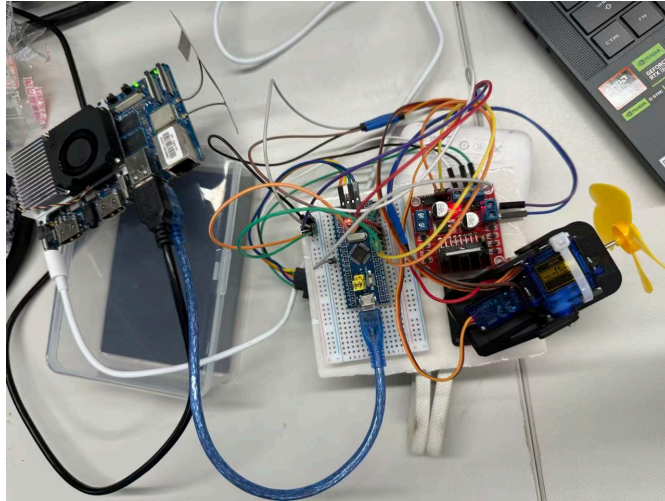


图 4: 系统整体实物

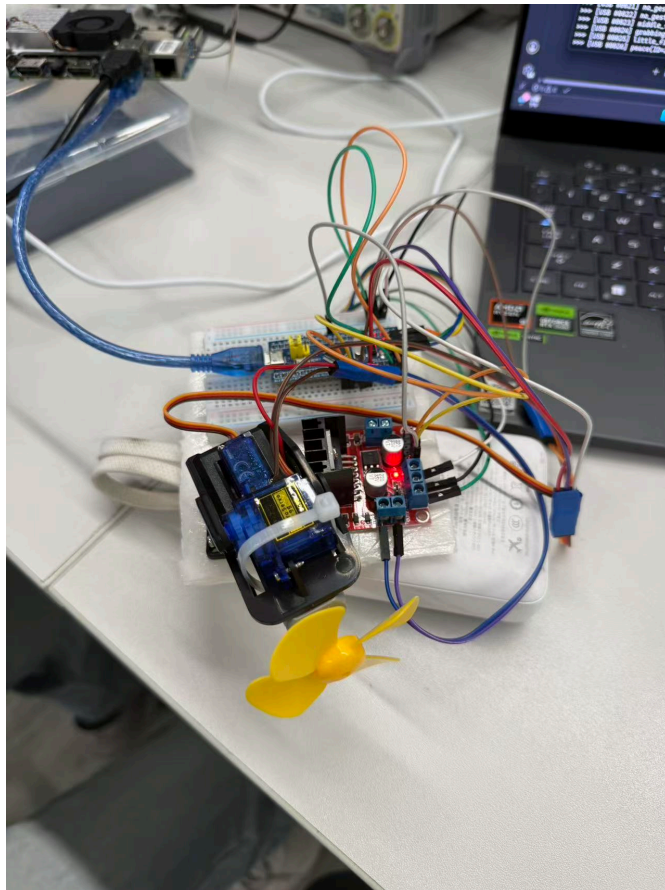


图 5: STM32 控制部分

系统运行演示视频: <https://www.bilibili.com/video/BV1dXNL6SELR/>

项目评估

成功之处

1. **架构设计合理**：昇腾 310B 负责计算密集的 AI 推理，STM32 负责实时控制，分工明确，充分发挥了各硬件的优势。
2. **通信协议简洁高效**：采用换行分隔的 JSON 格式，兼顾可读性和解析效率。轻量级字符串解析方案适合 STM32F103 这类资源受限的 MCU。在教师代码基础上仅需约 50 行修改即完成串口通信模块的集成。
3. **控制逻辑健壮**：开机/关机状态机设计避免了误触发，预判式折返扫描算法消除了舵机边界抖振。
4. **代码可维护性好**：模块化设计（gesture.c 独立模块）、注释完善、命名规范，便于后续扩展和团队协作。

不足之处

1. **手势识别对光照敏感**：在暗光或逆光环境下，手势检测准确率有所下降，需要更鲁棒的预处理或数据增强策略。
2. **串口通信缺乏校验机制**：当前 JSON 数据未加入校验和或序列号，异常情况下可能出现数据错乱而无法检测。
3. **风扇档位划分较粗**：仅 4 个固定档位，未实现无极调速，用户体验有提升空间。
4. **舵机扫描缺乏速度自适应**：左右和上下摆风速度为固定步长，未根据手势动态调整扫描速度。
5. **单手势控制限制**：每次仅识别一个手势，无法实现组合手势（如双手同时控制不同功能）。

个人反思与改进方向

通过本项目的开发实践，我在以下方面获得了深刻的体会与成长：

1. 边缘 AI 部署的工程挑战：将深度学习模型从训练环境迁移到边缘设备涉及模型转换、算子兼容性、精度验证、性能优化等多个环节。其中 ATC 模型转换是最容易出现算子不支持或精度损失的环节，需要对模型结构和硬件限制有充分了解。

2. 跨平台通信的可靠性设计：昇腾 310B（Linux+Python）与 STM32（裸机 C）之间的通信涉及不同平台的字节序、时序和协议约定。本项目采用的换行分隔 +JSON 格式是实用方案，但生产环境中应加入 CRC 校验、超时重传等机制以提高可靠性。

3. 中断与主循环的协作模式：STM32 开发中，中断服务函数仅做数据搬运和标志位置位，复杂业务逻辑交给主循环处理，这一模式有效避免了中断阻塞导致的 USB 通信异常。

4. 改进方向：

- 引入 Kalman 滤波或指数移动平均对识别结果进行时序平滑，减少单帧误识别导致的误触发
- 增加手势-动作映射的可配置性（如通过串口发送配置指令动态更新映射表）
- 实现风扇 PID 闭环调速（加入转速反馈传感器），替代当前的开环 PWM 控制
- 探索使用昇腾 310B 的额外算力运行手势追踪模型（如 MediaPipe），获取手部关键点以支持更精细的交互
- 增加 OTA 固件升级功能，通过昇腾 310B 端对 STM32 进行固件更新

附录

附录 A：完整代码清单

1. STM32 端主要源文件：

- Core/Src/main.c ——主程序（系统初始化、主循环手势控制逻辑）
- Core/Src/gesture.c ——手势数据结构与名称表定义
- Core/Inc/gesture.h ——手势模块头文件
- Core/Src/tim.c ——TIM3/TIM4 PWM 定时器配置
- Core/Src/gpio.c ——GPIO 初始化配置
- Core/Src/stm32f1xx_it.c ——中断服务函数
- USB_DEVICE/App/usbd_cdc_if.c ——USB CDC 接收回调与 JSON 解析
- USB_DEVICE/App/usbd_cdc_if.h ——CDC 接口头文件
- USB_DEVICE/App/usb_device.c ——USB 设备初始化

2. 昇腾 310B 端主要脚本：

- scripts/infer_om_camera.py ——OM 模型推理主程序（本人添加串口通信模块）
- hagrid_yolo/backends/acl_backend.py ——ACL NPU 推理后端（教师提供）
- hagrid_yolo/detector.py ——YOLO 检测器封装（教师提供）

- hagrid_yolo/preprocess.py ——图像预处理（教师提供）
- hagrid_yolo/postprocess.py ——检测框解码与 NMS 后处理（教师提供）
- hagrid_yolo/camera.py ——摄像头采集、推理循环、消抖发送（队友主导修改）
- hagrid_yolo/visualization.py ——检测框与标签绘制（教师提供）
- scripts/usb_sender.py ——USB CDC 串口发送模块（本人编写）

附录 B：设计图纸

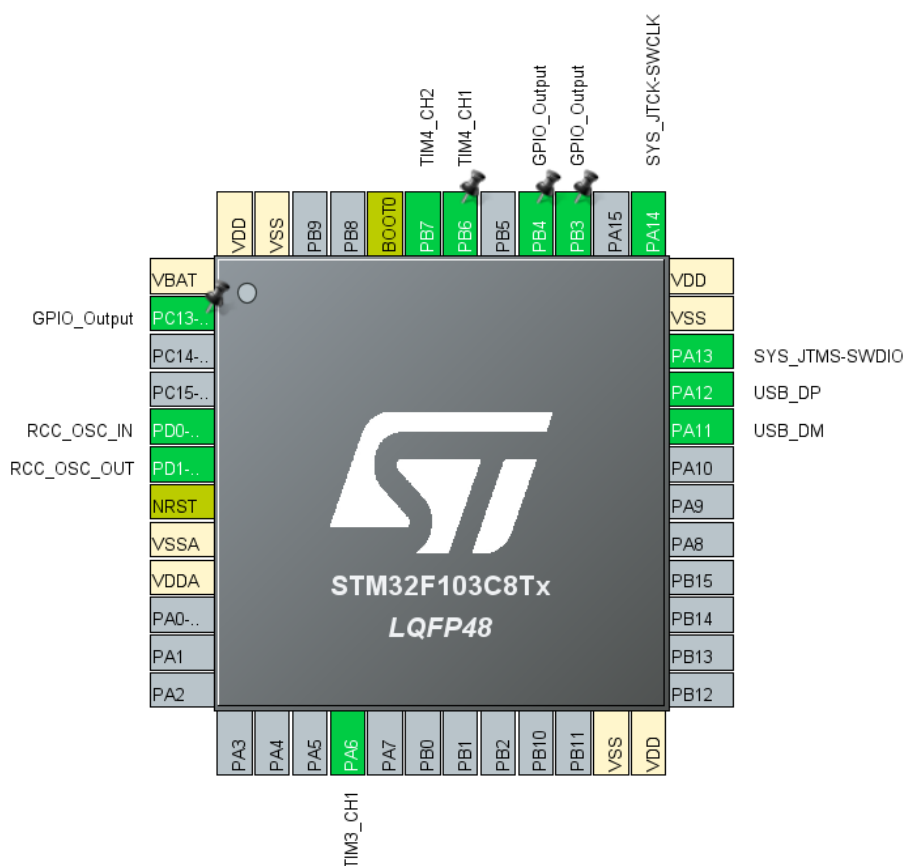


图 6: STM32CubeMX 管脚配置图

附录 C：参考资料

1. 周贤中. 昇腾 310B 实战——从入门到精通边缘计算与人工智能 [CP/OL]. <https://github.com/zhouxzh/Ascend310>.
2. 李志忠, 周贤中. 嵌入式系统设计及其项目开发 [M]. 西安: 西安电子科技大学出版社.

3. Wang A, Chen H, Liu L, et al. YOLOv10: Real-Time End-to-End Object Detection[J/OL]. arXiv preprint, 2024, arXiv:2405.14458.
4. Kapitanov A, Kvanchiani K, Nagaev A, et al. HaGRID——Hand Gesture Recognition Image Dataset[C]//Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision. 2024.
5. 华为技术有限公司. CANN 应用软件开发指南（Python）[EB/OL]. 华为昇腾社区.
6. STMicroelectronics. STM32F103x8/xB Reference Manual: RM0008[EB/OL]. 2021.